

Problema1

January 5, 2025

1 TP4 - Grupo 20

1.0.1 Dezembro, 4, 2024

Afonso Martins Campos Fernandes - A102940

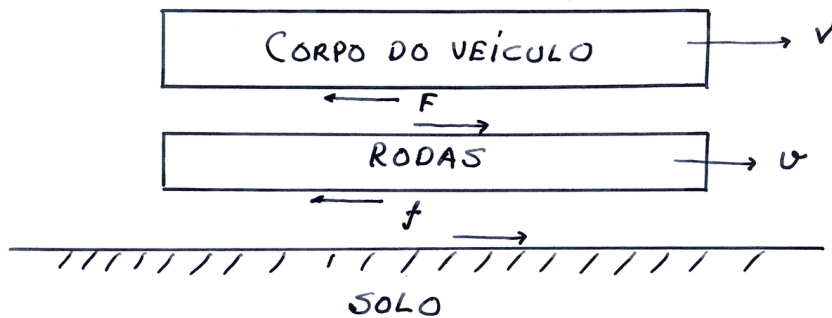
Luís Filipe Pinheiro Silva - A105530

1.1 ABS - Anti-Lock Braking System

No contexto do sistema de travagem ABS (“Anti-Lock Breaking System”), pretende-se construir um autómato híbrido que descreva o sistema e que possa ser usado para verificar as suas propriedades dinâmicas.

- A componente discreta do autómato contém os modos: **Start**, **Free**, **Stopping**, **Blocked**, e **Stopped**. No modo **Free** não existe qualquer força de travagem; no modo **Stopping** aplica-se a força de travagem alta; no modo **Blocked** as rodas estão bloqueadas em relação ao corpo mas o veículo desloca-se; no modo **Stopped** o veículo está imobilizado.
- A componente contínua do autómato usa variáveis contínuas V, v para descrever a **velocidade do corpo** do veículo em relação ao solo e a **velocidade linear das rodas** também em relação ao solo.
- Assume-se que o sistema de travagem exerce uma força de atrito nos travões proporcional à diferença das duas velocidades. A dinâmica contínua está descrita abaixo no bloco Equações de Fluxo.

Fazer: 1. Defina um autómato híbrido que descreva a dinâmica do sistema segundo as notas abaixo indicadas e com os “switchs” por si escolhidos. Os “switchs” (“jumps”) são uma componente de projeto deste trabalho; cabe ao aluno definir quais devem ser estas condições de modo a que o sistema tenha um comportamento desejável: imobilize-se depressa e não “derrape” muito. 2. Modele em lógica temporal linear LT propriedades que caracterizam o comportamento desejável do sistema. Nomeadamente 1. “o veículo imobiliza-se completamente em menos de t segundos” 2. “a velocidade V diminui sempre com o tempo”. 3. Construa o FOTS que descreve a discretização do modelo que definiu em a. e codifique em SMT’s 4. Codifique a verificação das propriedades temporais que definiu em b.



V - velocidade do corpo
em relação ao solo

v - velocidade linear das rodas
em relação ao solo

F - força de travagem (variável)

f - força de atrito ao solo
(constante)

Equações de Fluxo: 1. Durante a travagem não existe qualquer força no sistema excepto as forças de atrito. Quando uma superfície se desloca em relação à outra, a força de atrito é proporcional à força de compressão entre elas. 2. No contacto rodas/solo o atrito é constante porque a força de compressão é o peso; tem-se $f = a \cdot P$ sendo a a constante de atrito e P o peso. Ambos são fixos e independentes do modo. 3. No contacto corpo/rodas, a força de compressão é a força de travagem que aqui se assume como proporcional à diferença de velocidades $F = c \cdot (V - v)$. A constante de proporcionalidade c depende do modo: é elevada no modo **Stopping** e baixa nos outros. 4. Existe um atrito no contacto corpo/ar que é aproximado por uma constante positiva b . 5. As equações que traduzem a dinâmica do sistema são, em todos os modo excepto **Blocked**, $\dot{V} = -c \cdot (V - v) - b$ e $\dot{v} = -a \cdot P + c \cdot (V - v)$, no modo **Blocked**, a dinâmica do sistema é regida por $(V = v) \wedge (\dot{V} = -a \cdot P - b)$ 6. Tanto no modo **Blocked** como no modo **Free** existe um “timer” que impede que se permaneça nesses modo mais do que τ segundos. Os **jumps**(V, v, t, V', v', t') com origem nesses modos devem forçar esta condição. 7. No instante inicial assume-se $V = v = V_0$, em que a velocidade V_0 é o “input” do problema.

1.2 Alínea a)

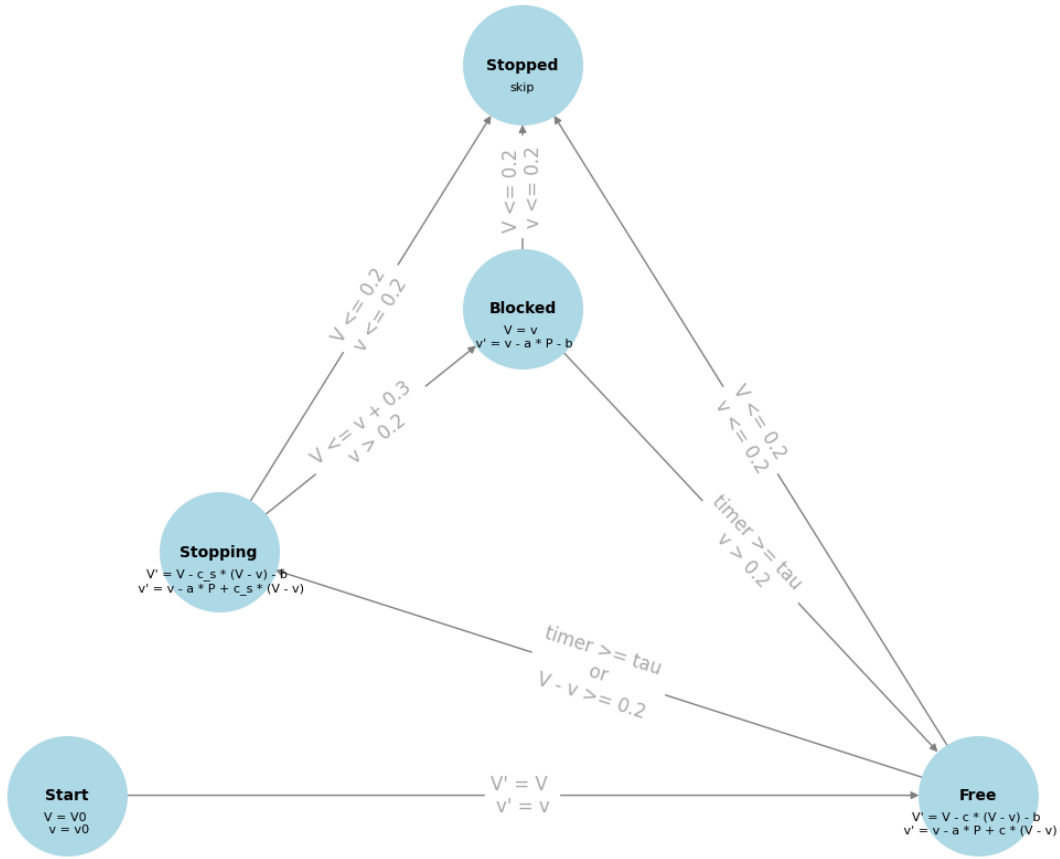
Utilizamos o matplotlib.pyplot e o networkx para desenhar o autómato híbrido que descreve o sistema ABS.

```
[29]: import matplotlib.pyplot as plt
import networkx as nx

# Criação do grafo usando NetworkX
G = nx.DiGraph()

# Adicionando nós
G.add_node('Start', annotation="V = V0 \n v = v0")
```


ABS - “Anti-Lock Breaking System”



1.3 Objetivo:

Para esta alinea temos que construir um automato híbrido que retrate o comportamento dinâmico de um sistema de travagem ABS. O autômato deve capturar componentes discretas quanto as contínuas do sistema, e também deve representar as condições de transição entre os seus diferentes modos operacionais.

1.4 Implementação:

Ferramentas Utilizadas:

- **Matplotlin** : Para a visualização gráfica do autômato criado.
- **NetworkX**: Para criar e manipular o grafo que represente os estados e as transições do sistema.

Estrutura: O autômato híbrido criado é formado por cinco modos discretos, sendo estes:

1. **Start**: é o estado inicial em que $V = V_0$ e $v = v_0$.
2. **Free**: O veículo está em movimento livre sem aplicação de força de travagem.

3. **Stopping**: Estado em que a força de travagem elevada é aplicada.
4. **Blocked**: As rodas do veículo ficam bloqueadas em relação ao corpo, apesar do veículo estar em movimento.
5. **Stopped**: Estado em que o veículo fica completamente imobilizado.

Para existir as transições entre estados ou jumps entre estados é preciso existir algumas condições de transição, como:

1. **Start** -> **Free** : $V' = V$ e $v' = v$.
2. **Free** -> **Stopping** : $\text{timer} \geq \tau$ ou $V - v \geq 0.2$
3. **Free** -> **Stopped** : $V \leq 0.2$ e $v \leq 0.2$
4. **Stopping** -> **Stopped** : $V \leq 0.2$ e $v \leq 0.2$
5. **Blocked** -> **Free** : $\text{timer} \geq \tau$ e $v > 0.2$
6. **Blocked** -> **Stopped** : $V \leq 0.2$ e $v \leq 0.2$

A dinâmica contínua do sistema é regida pelas seguintes equações:

- Nos estados **Free**, **Stopping** e **Stopped**:

$$\dot{V} = -c \cdot (V - v) - b$$

$$\dot{v} = -a \cdot P + c \cdot (V - v)$$

- No modo **Blocked**:

$$V = v, \quad \dot{V} = -a \cdot P - b$$

1.5 Alínea c)

Começar por criar as funções das variáveis, do estado inicial e das transições

1.5.1 Função states(i):

i - valor do traço

```
[30]: from pysmt.shortcuts import *
      from z3 import *

      #Modes, (START, FREE, STOPPING, BLOCKED, STOPPED) = \
      ↪ EnumSort('Modes', ('START', 'FREE', 'STOPPING', 'BLOCKED', 'STOPPED'))

      def states(i):
          state = {}

          state['V'] = Real('V'+str(i))
```

```

state['v'] = Real('v'+str(i))
state['m'] = Const('m'+str(i),Modes)
state['t'] = Real('t'+str(i))
state['timer'] = Real('timer'+str(i))

return state

```

1.5.2 Função init(state,v_init):

state - estado atual v_init - velocidade inicial

```

[31]: def init(state,v_init):
        return And(
            state['V'] == v_init,
            state['v'] == v_init,
            state['m'] == START,
            state['t'] == 0,
            state['timer'] == 0
        )

```

1.5.3 Função trans(curr,next):

curr - estado atual next - próximo estado

```

[32]: c,c_s,a,P,b,tau = 0.1,1,0.001,1300,0.005,0.1 # Variaveis para asfalto seco
# c,c_s,a,P,b,tau = 0.15,1,0.01,1300,0.005,0.1 # Variaveis para asfalto molhado

def trans(curr,prox):

    t0 = And(prox['V'] == curr['v'], #Transição do Start para o Free
        prox['v'] == curr['v'],
        prox['m'] == FREE,
        curr['m'] == START,
        prox['t'] == curr['t'],
        prox['timer'] == 0,
        curr['timer'] == 0
    )

    t1 = And(prox['V'] == curr['V'] + ((-c)*(curr['V'] - curr['v']) - b),
        ↪#Transição do Free para o Free
        prox['v'] == curr['v'] + ((-a)*P + c*(curr['V'] - curr['v'])),
        prox['m'] == FREE,
        curr['m'] == FREE,
        prox['t'] == curr['t'] + 0.1,
        prox['timer'] == curr['timer'] + 0.1,
        prox['timer'] <= tau,
        curr['v'] > 0.2
    )

```

```

t2 = And(prox['V'] == curr['V'], #Transição do Free para o Stopping
        prox['v'] == curr['v'],
        prox['m'] == STOPPING,
        curr['m'] == FREE,
        prox['t'] == curr['t'],
        prox['timer'] == 0,
        Or(
            curr['timer'] >= tau,
            curr['V'] - curr['v'] >= 0.1
        ))

t3 = And(prox['V'] == curr['V'] - c_s*(curr['V'] - curr['v']) - b, #
→Transição do Stopping para o Stopping
        prox['v'] == curr['v'] - a*P + c_s*(curr['V'] - curr['v']),
        prox['m'] == STOPPING,
        curr['m'] == STOPPING,
        prox['t'] == curr['t'] + 0.1,
        prox['timer'] == curr['timer'],
        curr['V'] > curr['v'] + 0.1
    )

t4 = And(prox['V'] == 0, #Transição do Stopping para o Stopped
        prox['v'] == 0,
        prox['m'] == STOPPED,
        curr['m'] == STOPPING,
        prox['t'] == curr['t'],
        prox['timer'] == 0,
        curr['V'] <= 0.2,
        curr['v'] <= 0.2
    )

t5 = And(prox['V'] == curr['V'], #Transição do Stopping para o Blocked
        prox['v'] == curr['v'],
        prox['m'] == BLOCKED,
        curr['m'] == STOPPING,
        prox['t'] == curr['t'],
        prox['timer'] == 0,
        curr['V'] <= curr['v'] + 0.1,
        curr['v'] > 0.2
    )

t6 = And(curr['V'] == curr['v'], #Transição do Blocked para o Blocked
        prox['V'] == curr['V'] - a*P - b,
        prox['m'] == BLOCKED,
        curr['m'] == BLOCKED,
        prox['t'] == curr['t'] + 0.1,

```

```

        prox['timer'] == curr['timer'] + 0.1,
        prox['timer'] <= tau
    )

t7 = And(prox['V'] == curr['V'], #Transição do Blocked para o Free
        prox['v'] == curr['v'],
        prox['m'] == FREE,
        curr['m'] == BLOCKED,
        prox['t'] == curr['t'],
        prox['timer'] == 0,
        curr['timer'] >= tau,
        curr['v'] > 0.2
    )

t8 = And(prox['V'] == 0, #Transição do Blocked para o Stopped
        prox['v'] == 0,
        prox['m'] == STOPPED,
        curr['m'] == BLOCKED,
        prox['t'] == curr['t'],
        prox['timer'] == 0,
        curr['v'] <= 0.2,
        curr['V'] <= 0.2
    )

t9 = And(prox['V'] == curr['V'], # Transição do Stopped para o Stopped
        prox['v'] == curr['v'],
        prox['m'] == STOPPED,
        curr['m'] == STOPPED,
        prox['t'] == curr['t'] + 0.1,
        prox['timer'] == curr['timer']
    )

t10 = And(prox['V'] == 0, #Transição do Free para o Stopped
        prox['v'] == 0,
        prox['m'] == STOPPED,
        curr['m'] == FREE,
        prox['t'] == curr['t'],
        prox['timer'] == 0,
        curr['v'] <= 0.2,
        curr['V'] <= 0.2
    )

return Or(t0,t1,t2,t3,t4,t5,t6,t7,t8,t9,t10)

```

1.5.4 Função gera_fots(states,init,trans,k,v_init)

states - função que gera as variáveis init - função que verifica se um certo estado é estado inicial trans - função que verifica se existe transição entre dois estados k - número de estados

a verificar v_{init} - velocidade inicial do problema

```
[40]: def gera_fots(states,init,trans,k,v_init):

    solver = Solver()

    estados = [states(i) for i in range(k)]

    solver.add(init(estados[0],v_init))

    for i in range(k-1):
        solver.add(trans(estados[i],estados[i+1]))

    check = solver.check()

    if check == sat:
        print("Is solvable")
        m = solver.model()

        for i in range(k):
            print("Passo:", i)
            for v in estados[i]:
                res = m[estados[i][v]]

                if res.sort() != RealSort():
                    print(v, '=', res)
                else:
                    print(v, '=', float(res.numerator_as_long())/float(res.
→denominator_as_long()))
            else:
                print("Not solvable")

gera_fots(states,init,trans,60,30)
```

Is solvable

Passo: 0

V = 30.0

v = 30.0

m = START

t = 0.0

timer = 0.0

Passo: 1

V = 30.0

v = 30.0

m = FREE

t = 0.0

timer = 0.0

Passo: 2

V = 29.995
v = 28.7
m = FREE
t = 0.1
timer = 0.1
Passo: 3
V = 29.995
v = 28.7
m = STOPPING
t = 0.1
timer = 0.0
Passo: 4
V = 28.695
v = 28.695
m = STOPPING
t = 0.2
timer = 0.0
Passo: 5
V = 28.695
v = 28.695
m = BLOCKED
t = 0.2
timer = 0.0
Passo: 6
V = 27.39
v = 27.54
m = BLOCKED
t = 0.3
timer = 0.1
Passo: 7
V = 27.39
v = 27.54
m = FREE
t = 0.3
timer = 0.0
Passo: 8
V = 27.4
v = 26.225
m = FREE
t = 0.4
timer = 0.1
Passo: 9
V = 27.4
v = 26.225
m = STOPPING
t = 0.4
timer = 0.0
Passo: 10

```
V = 26.22
v = 26.1
m = STOPPING
t = 0.5
timer = 0.0
Passo: 11
V = 26.095
v = 24.92
m = STOPPING
t = 0.6
timer = 0.0
Passo: 12
V = 24.915
v = 24.795
m = STOPPING
t = 0.7
timer = 0.0
Passo: 13
V = 24.79
v = 23.615
m = STOPPING
t = 0.8
timer = 0.0
Passo: 14
V = 23.61
v = 23.49
m = STOPPING
t = 0.9
timer = 0.0
Passo: 15
V = 23.485
v = 22.31
m = STOPPING
t = 1.0
timer = 0.0
Passo: 16
V = 22.305
v = 22.185
m = STOPPING
t = 1.1
timer = 0.0
Passo: 17
V = 22.18
v = 21.005
m = STOPPING
t = 1.2
timer = 0.0
Passo: 18
```

```
V = 21.0
v = 20.88
m = STOPPING
t = 1.3
timer = 0.0
Passo: 19
V = 20.875
v = 19.7
m = STOPPING
t = 1.4
timer = 0.0
Passo: 20
V = 19.695
v = 19.575
m = STOPPING
t = 1.5
timer = 0.0
Passo: 21
V = 19.57
v = 18.395
m = STOPPING
t = 1.6
timer = 0.0
Passo: 22
V = 18.39
v = 18.27
m = STOPPING
t = 1.7
timer = 0.0
Passo: 23
V = 18.265
v = 17.09
m = STOPPING
t = 1.8
timer = 0.0
Passo: 24
V = 17.085
v = 16.965
m = STOPPING
t = 1.9
timer = 0.0
Passo: 25
V = 16.96
v = 15.785
m = STOPPING
t = 2.0
timer = 0.0
Passo: 26
```

```
V = 15.78
v = 15.66
m = STOPPING
t = 2.1
timer = 0.0
Passo: 27
V = 15.655
v = 14.48
m = STOPPING
t = 2.2
timer = 0.0
Passo: 28
V = 14.475
v = 14.355
m = STOPPING
t = 2.3
timer = 0.0
Passo: 29
V = 14.35
v = 13.175
m = STOPPING
t = 2.4
timer = 0.0
Passo: 30
V = 13.17
v = 13.05
m = STOPPING
t = 2.5
timer = 0.0
Passo: 31
V = 13.045
v = 11.87
m = STOPPING
t = 2.6
timer = 0.0
Passo: 32
V = 11.865
v = 11.745
m = STOPPING
t = 2.7
timer = 0.0
Passo: 33
V = 11.74
v = 10.565
m = STOPPING
t = 2.8
timer = 0.0
Passo: 34
```

```
V = 10.56
v = 10.44
m = STOPPING
t = 2.9
timer = 0.0
Passo: 35
V = 10.435
v = 9.26
m = STOPPING
t = 3.0
timer = 0.0
Passo: 36
V = 9.255
v = 9.135
m = STOPPING
t = 3.1
timer = 0.0
Passo: 37
V = 9.13
v = 7.955
m = STOPPING
t = 3.2
timer = 0.0
Passo: 38
V = 7.95
v = 7.83
m = STOPPING
t = 3.3
timer = 0.0
Passo: 39
V = 7.825
v = 6.65
m = STOPPING
t = 3.4
timer = 0.0
Passo: 40
V = 6.645
v = 6.525
m = STOPPING
t = 3.5
timer = 0.0
Passo: 41
V = 6.52
v = 5.345
m = STOPPING
t = 3.6
timer = 0.0
Passo: 42
```

```
V = 5.34
v = 5.22
m = STOPPING
t = 3.7
timer = 0.0
Passo: 43
V = 5.215
v = 4.04
m = STOPPING
t = 3.8
timer = 0.0
Passo: 44
V = 4.035
v = 3.915
m = STOPPING
t = 3.9
timer = 0.0
Passo: 45
V = 3.91
v = 2.735
m = STOPPING
t = 4.0
timer = 0.0
Passo: 46
V = 2.73
v = 2.61
m = STOPPING
t = 4.1
timer = 0.0
Passo: 47
V = 2.605
v = 1.43
m = STOPPING
t = 4.2
timer = 0.0
Passo: 48
V = 1.425
v = 1.305
m = STOPPING
t = 4.3
timer = 0.0
Passo: 49
V = 1.3
v = 0.125
m = STOPPING
t = 4.4
timer = 0.0
Passo: 50
```

```
V = 0.12
v = 0.0
m = STOPPING
t = 4.5
timer = 0.0
Passo: 51
V = 0.0
v = 0.0
m = STOPPED
t = 4.5
timer = 0.0
Passo: 52
V = 0.0
v = 0.0
m = STOPPED
t = 4.6
timer = 0.0
Passo: 53
V = 0.0
v = 0.0
m = STOPPED
t = 4.7
timer = 0.0
Passo: 54
V = 0.0
v = 0.0
m = STOPPED
t = 4.8
timer = 0.0
Passo: 55
V = 0.0
v = 0.0
m = STOPPED
t = 4.9
timer = 0.0
Passo: 56
V = 0.0
v = 0.0
m = STOPPED
t = 5.0
timer = 0.0
Passo: 57
V = 0.0
v = 0.0
m = STOPPED
t = 5.1
timer = 0.0
Passo: 58
```



```

V = 0.0
v = 0.0
m = STOPPED
t = 5.2
timer = 0.0
Passo: 59
V = 0.0
v = 0.0
m = STOPPED
t = 5.3
timer = 0.0

```

1.5.5 1) Estrutura:

1.5.6 1.1) Componentes:

- Variáveis de Estado:
 - V e v: Variáveis de Estado que definem a velocidade do corpo e rodas, respectivamente;
 - t, timer: Variáveis de Estado que definem o tempo decorrido desde o início do processo de travagem e há quanto tempo o veículo encontra-se no estado atual;
 - m: Estado do veículo (Free, Stopping, Blocked ou Stopped);
- Constantes do Autómato:
 - c e c_s : constante de proporcionalidade no estado Free e Stopping, respectivamente; tau
 - a e P : constante de atrito e peso do veículo, respectivamente;
 - b : constante do atrito corpo/ar;
 - tau : tempo máximo que o veículo pode permanecer no estado Free ou Blocked sem trocar de estado;

1.5.7 1.2) Funções Principais:

- def states(i):

Esta função gera um novo estado do sistema com variáveis simbólicas, sendo os seus parâmetros com uma lista de variáveis, prefixo do estado, índice e o número de bits.

- def estado_inicial(state, v_init):

Esta função define as condições iniciais do sistema, inicializando todas as variáveis com os seus valores iniciais.

- def transicao(curr, prox):

A função representada define as regras para a transição de estados, implementa a lógica principal do algoritmo.

1.5.8 2) Implementação

`gera_trace(states,init, trans, 60, 30)`

Para testarmos se o sistema está funcionar corretamente utilizamos a função `gera_fots` para o mesmo ser testado. A função `gera_fots` recebe os parametros: função geradora de variáveis de estados(`states`), o estado inicial(`init`), a função transição(`trans`), o número de passos a serem efetuados(60) e a velocidade inicial do veículo(8).

A função `gera_fots(states,init,trans,60,30)` ao ser executada devolve os valores de todas as variáveis em cada estado e as transições de cada estado.

1.6 Alínea b)

Modelar em lógica temporal linear LT propriedades que caracterizam o comportamento desejável do sistema.

```
[34]: def imobiliza_em_t(state):  
        return Implies(state['t'] >= time, state['m'] == STOPPED)  
  
def velocidade_diminui_sempre(curr,prox):  
    return Implies(curr['t'] < prox['t'], curr['V'] > prox['V'])
```

1.6.1 1) Propriedades Desejáveis:

1.6.2 1.1) O veículo imobiliza-se completamente em menos de t segundos:

A função `imobiliza_em_t(state,time)` verifica que se passaram `time` segundos, o veículo encontra-se imobilizado. Pode ser representado no formato:

“If `t >= time`, then `m = STOPPED`”

1.6.3 1.2) A velocidade V diminui sempre com o tempo:

A função `velocidade_diminui_sempre(curr,prox)` verifica que entre dois estados consecutivos, a velocidade do corpo diminui sempre. Pode ser representado no formato:

“If `curr['t'] < prox['t']`, then `curr['V'] > prox['V']`”

1.7 Alínea d)

1.7.1 Função `testa_imobiliza(states,init,trans,inv,k,time)`

`states` - função que gera as variáveis `init` - função que verifica se um certo estado é estado inicial
`trans` - função que verifica se existe transição entre dois estados `inv` - invariante a testar
`k` - número de estados a verificar `time` - tempo em que o veículo deve se imobilizar

1.7.2 Função `testa_diminui(states,init,trans,inv,k)`

`states` - função que gera as variáveis `init` - função que verifica se um certo estado é estado inicial
`trans` - função que verifica se existe transição entre dois estados `inv` - invariante a testar
`k` - número de estados a verificar

```
[35]: def testa_imobiliza(states,init,trans,inv,k,time):
    for k in range(1,k+1):
        s = Solver()

        estados = [states(i) for i in range(k)]

        s.add(init(estados[0],30))

        for i in range(k-1):
            s.add(trans(estados[i],estados[i+1]))

        s.add(Not(inv(estados[k-1],time)))

        if s.check() == sat:
            print("Propriedade inválida")
            return

        print ("Propriedade pode ser válida")

testa_imobiliza(states,init,trans,imobiliza_em_t,60,7)
```

Propriedade pode ser válida

```
[ ]: def testa_diminui(states,init,trans,inv,k):
    for k in range(1,k+1):
        s = Solver()

        estados = [states(i) for i in range(k)]

        s.add(init(estados[0],20))
        aux=[]
        for i in range(k-1):
            s.add(trans(estados[i],estados[i+1]))
            aux.append(Not(inv(estados[i],estados[i+1])))

        s.add(Or(aux))

        if s.check() == sat:
            print("Propriedade inválida")
            return

        print ("Propriedade pode ser válida")

testa_diminui(states,init,trans,velocidade_diminui_sempre,40)
```

Propriedade inválida

1.7.3 1) Verificação:

Para a verificação do sistema, são feitos os seguintes passos:

1. Iniciação:

São gerados diferentes estados, definidos símbolos de entrada e é estabelecido o estado inicial.

2. Transições:

São definidas as transições e as propriedades desejáveis.

3. Implementação da Verificação:

Utilizamos a função `testa_imobiliza(states,init,trans,inv,k,time)` que nos mostra de forma simples e eficaz se a propriedade “O veículo imobiliza-se completamente em menos de t segundos” é possível neste sistema.

Utilizamos a função `testa_diminui(states,init,trans,inv,k)` que nos mostra de forma simples e eficaz se a propriedade “A velocidade V diminui sempre com o tempo” é possível neste sistema.